Infraestructura y operación

Inventario histórico: 2026-10-05

Estado verificado después del inventario: staging eliminado/no desplegado al 2026-10-09; producción no desplegada.

Fuentes: inventario AWS verificado el 2026-10-05 para el estado observado entonces; `infra/cdk`, `docker-compose.yml`, `infra/docker` y `.github/workflows` para la configuración declarada.

El inventario del 2026-10-05 observó staging en la cuenta `075566976700`, región `us-east-2`: los stacks `PukStagingFoundation` y `PukStagingApplication`, una tarea ECS en EC2 `t3.small`, RDS independiente del host, la IPv4 efímera `18.222.63.51` y el secreto runtime `puk/staging/runtime`. Al 2026-10-09 staging está eliminado/no desplegado y producción no está desplegada. Los datos del inventario son referencias históricas, no evidencia de recursos activos. No se inspeccionaron datos de RDS.

Diferencia observada históricamente: la revisión de tarea 14 usaba S3; la IaC local selecciona Drive. El bucket `EvidenceBucket` estaba referenciado por esa revisión. La consulta de versiones del 2026-10-05 devolvió cero versiones y marcadores de borrado, sin paginación pendiente; no habilita su eliminación mientras siga siendo un consumidor activo. La sincronización espera configuración segura y validación de los recursos de Google de pruebas. No se han eliminado datos ni recursos como parte de esta sincronización. Producción no tiene stacks CDK propios.

Ambientes

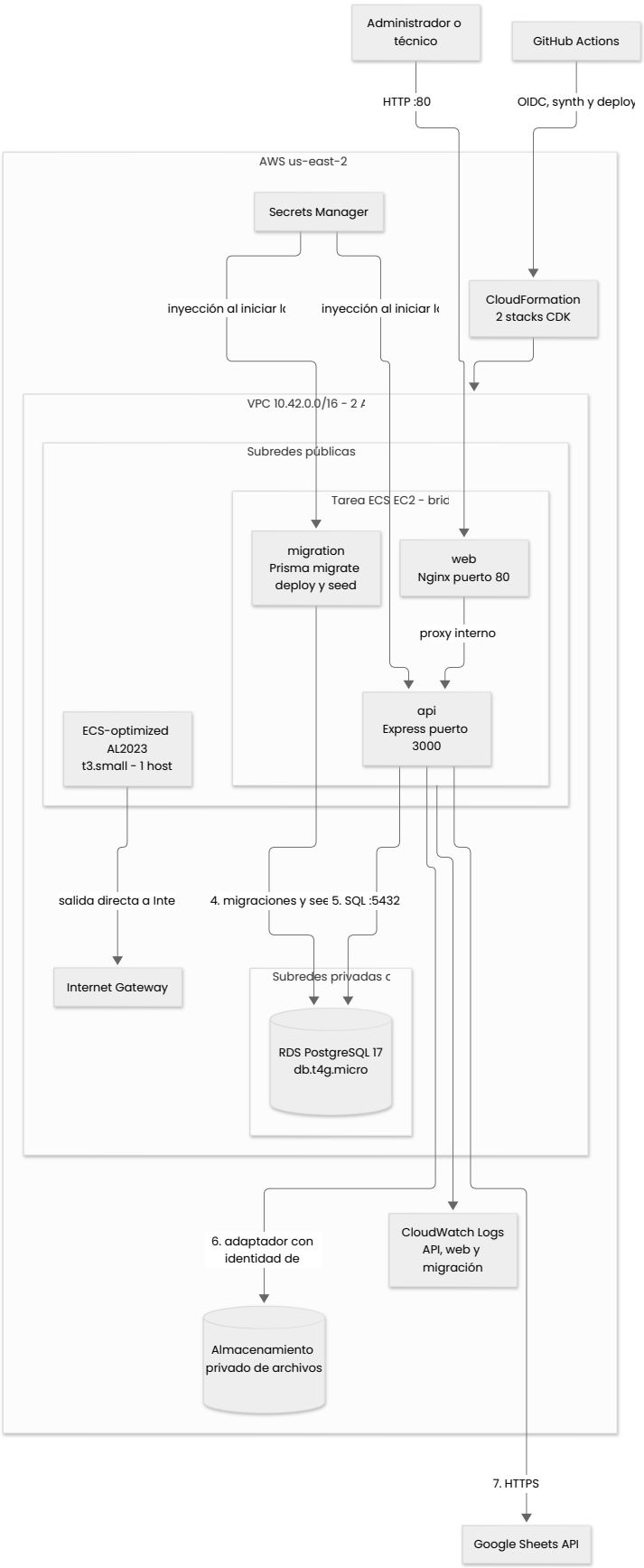
Ambiente	Implementación actual	Persistencia	Entrada
Desarrollo con hot reload	Vite y API en el host; PostgreSQL y MinIO en Docker Compose	Volúmenes locales	`http://localhost:5173` y `http://localhost:3000`
Desarrollo compilado	Contenedores `web`, `api`, `postgres`, `minio` y `minio-init`	Volúmenes locales	`http://localhost:4173` y `http://localhost:3000`
Staging	No desplegado al 2026-10-09; la IaC describe ECS sobre EC2, RDS, almacenamiento y logs	Por verificar	Sin endpoint activo
Producción	No implementado como IaC en la rama actual	Por definir	Por definir

Arquitectura de staging en AWS: topología declarada por CDK

El diagrama y las secciones siguientes describen la infraestructura declarada en CDK, no recursos activos. Staging está eliminado/no desplegado al 2026-10-09; verificar el diff y el estado real de AWS antes de cualquier operación.

Vista de despliegue

> **Topología declarada por IaC:** los detalles corresponden al diseño en código. El inventario histórico observó S3; Drive es el proveedor seleccionado por la IaC local y requiere configuración y validación antes de un redeployment.



El diagrama muestra la topología definida por CDK: subredes públicas, privadas para aplicación y privadas aisladas, una por zona de disponibilidad; un Auto Scaling Group fijo con una instancia EC2 `t3.small` en subred pública y una IPv4 efímera; HTTP directo por el puerto 80;

sin NAT Gateway, Elastic IP, ALB ni CloudFront. Nginx enruta internamente al API y RDS queda en subredes privadas aisladas. Esto no confirma la existencia actual de esos recursos.

Recorrido de una solicitud

1. El navegador accede a `http://<ipv4-publica>` directamente.
2. El security group acepta sólo HTTP TCP/80 desde Internet; no expone SSH ni el puerto 3000.
3. Nginx sirve el frontend Vue, aplica fallback a `index.html` y enruta `/api/` , `/health/` , `/docs/` y `/metrics` al API por la red interna de contenedores.
4. Express procesa API, salud, métricas y OpenAPI.

Esta entrada no usa TLS: las credenciales y cookies de sesión viajan sin cifrado. La IPv4 es efímera y puede cambiar al detener/reemplazar el host; no existe un dominio estable ni CDN, protección de borde, terminación TLS o caché CDN. No debe usarse para datos reales ni producción.

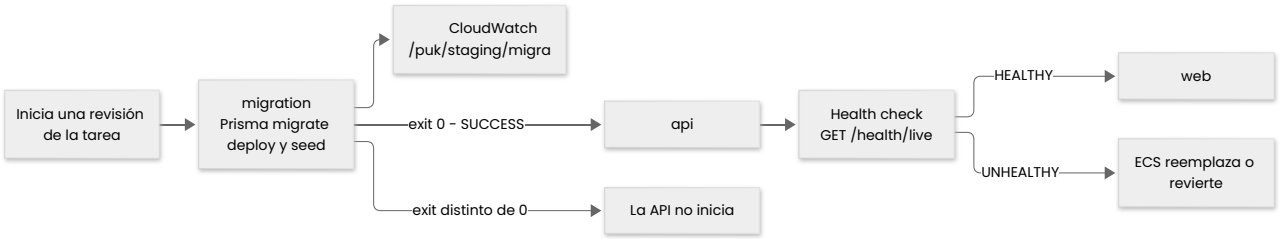
Red y límites de confianza

Zona	Recursos	Entrada permitida	Salida relevante
Pública	Host ECS EC2 con IPv4 pública efímera	TCP/80 desde cualquier IPv4	Internet Gateway, RDS y APIs AWS
Privada aislada	RDS PostgreSQL	TCP/5432 sólo desde el security group de la aplicación	Sin ruta directa a Internet
Fuera de la VPC	Google Drive, Secrets Manager y CloudWatch	Mediante APIs administradas	Según el servicio

El host EC2 recibe IPv4 pública efímera para conexiones entrantes y salientes sin NAT Gateway ni Elastic IP. Su security group sólo permite TCP/80 desde Internet; el API en el puerto 3000 no se publica. La API requiere salida HTTPS para Google Sheets, Google Drive y AWS. La tarea usa modo `bridge` y conserva sus roles ECS propios; IMDSv2 es obligatorio y el hop limit es 1 para impedir que contenedores alcancen las credenciales del instance profile. La instancia no recibe permisos de Secrets Manager ni de S3. Esta configuración evita el costo horario y de procesamiento del NAT, aunque la IPv4 pública sigue teniendo costo y la salida no pasa por un NAT controlado.

Tarea ECS sobre EC2 y orden de arranque declarado

> **Orden definido por IaC:** al iniciarse una tarea, el contenedor de migración termina antes de que pueda iniciar la API; no implica que haya una tarea activa actualmente.



Los tres contenedores comparten la tarea; ECS la ejecuta en modo `bridge` dentro del host EC2. Nginx se vincula al contenedor `api` por el alias `api`. Nginx publica el puerto de host 80; el API también mapea el puerto de host 3000, pero el security group bloquea el ingreso externo a ese puerto. Estas son propiedades de la definición CDK, no una observación del entorno actual:

- `migration` no es esencial. Cada vez que se inicia una tarea nueva, ejecuta `prisma migrate deploy` y después `prisma/seed.ts`; la API depende de que termine con éxito (`SUCCESS`). `migrate deploy` aplica sólo las migraciones pendientes presentes en `prisma/migrations/`. Un cambio únicamente en `prisma/schema.prisma` no modifica la base de datos: hay que crear y versionar la migración correspondiente. Las imágenes de API y migración generan el cliente Prisma durante el build Docker con una URL de base de datos ficticia de build; esa generación no se conecta a la base de datos de producción ni ejecuta migraciones. Un despliegue sólo de código puede iniciar el contenedor de migración, pero no cambia el esquema si no hay migraciones pendientes.
- `api` es esencial y expone el puerto 3000. Su health check interno consulta `/health/live`.
- `web` es esencial y expone el puerto 80. Sólo inicia cuando la API está saludable.
- El servicio usa `desiredCount: 1` en un ASG de capacidad fija (mínimo/máximo/deseado 1). El despliegue usa 0 % mínimo y 100 % máximo para detener la tarea anterior antes de iniciar la nueva: hay una interrupción breve durante cada despliegue y no se garantiza alta disponibilidad ante falla del host.
- El host es `t3.small` con CPU credits en modo `standard`: se limita el costo inesperado de créditos surplus, a cambio de que el rendimiento de CPU pueda bajar al agotarse el balance bajo carga sostenida.

La migración dentro de cada tarea mantiene el orden de arranque, pero con más de una tarea podría ejecutar migraciones o seed de forma concurrente. Antes de escalar horizontalmente se debe convertir la migración en un job único por despliegue o comprobar explícitamente su idempotencia y bloqueo.

Persistencia

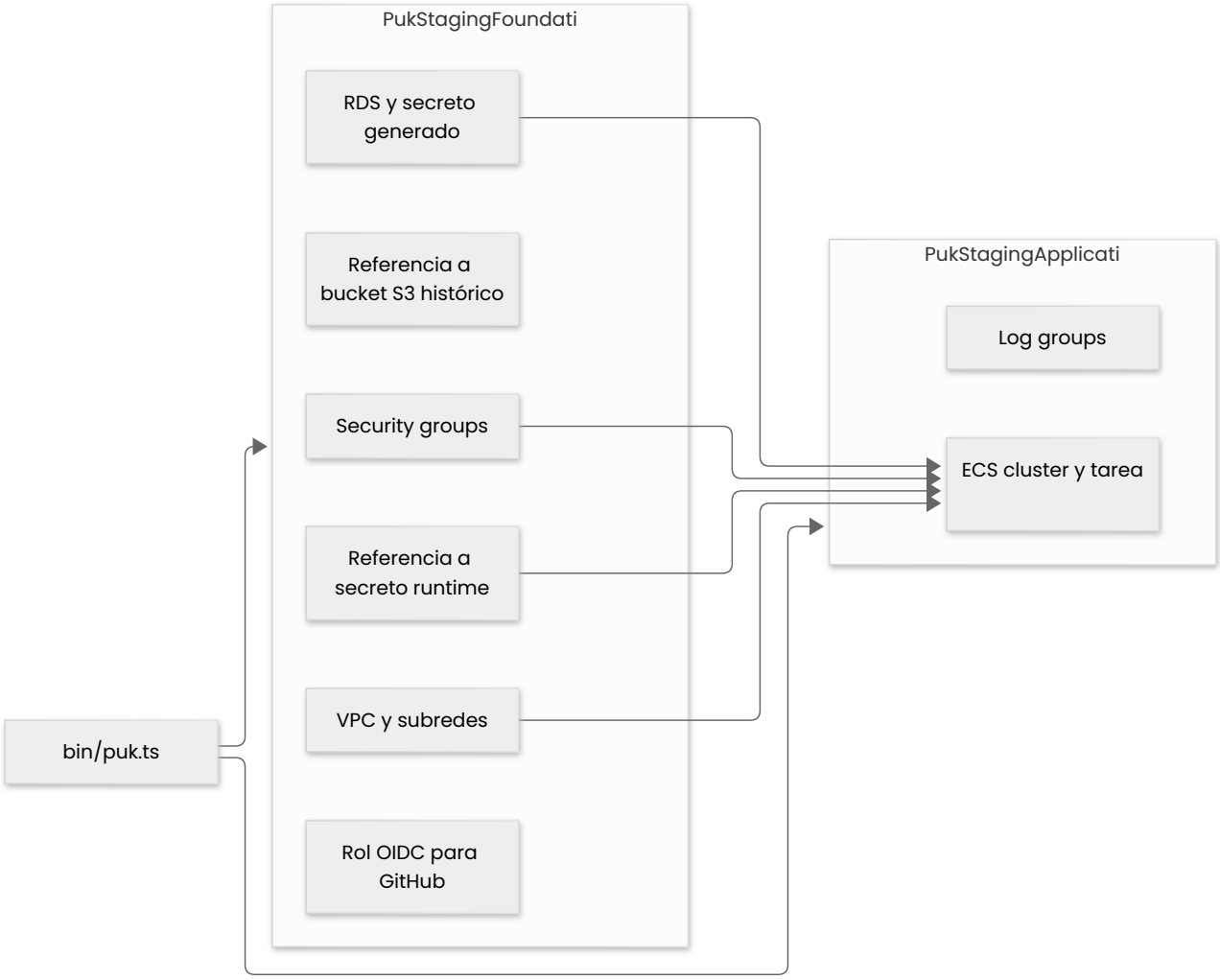
Recurso	Configuración declarada / diferencia histórica	Política IaC al eliminar en el futuro
RDS PostgreSQL 17	`db.t4g.micro`, 20 GiB, autoscaling hasta 100 GiB, cifrado, Single-AZ, 1 día de backup automático, sin acceso público	`RETAIN` y deletion protection
Almacenamiento de archivos	Históricamente S3; Drive seleccionado por la IaC local y pendiente de configuración para redeploy	Conservar archivos y bucket hasta validar transición

Recurso	Configuración declarada / diferencia histórica	Política IaC al eliminar en el futuro
Logs	Grupos separados para migración, API y web; retención de un mes	`RETAIN`

El objetivo de producto es usar una unidad compartida de Google Drive para evidencias, firmas y reportes; PostgreSQL conserva referencias y metadatos. La IaC local selecciona `STORAGE_PROVIDER=drive`, no declara buckets ni permisos S3 nuevos, y requiere los tres campos `GOOGLE_DRIVE_*`. La última revisión de tarea observada históricamente usaba S3; no hay una revisión desplegada actualmente. Conservar bucket, referencias y permisos necesarios hasta validar el reemplazo; cambiar de proveedor no migra archivos existentes. La política de respaldo independiente sigue pendiente antes de producción.

Stacks CDK

> **Dependencias declaradas entre stacks:** revisar los exports consumidos antes de desplegar cualquier limpieza o redeployment.



`PukStagingApplication` recibe referencias del stack Foundation, por lo que CDK crea dependencias y exports de CloudFormation. Foundation concentra recursos con estado o compartidos; Application concentra cómputo. Esta separación reduce el riesgo al reemplazar la capa de aplicación, aunque el stack foundation no activa `terminationProtection` a nivel de CloudFormation.

La IaC conserva una referencia a un security group legado del ALB para compatibilidad entre stacks. No hay ALB ni NAT Gateway en el inventario regional consultado. Tanto esa referencia como los exports S3 requieren limpieza ordenada: preservar el recurso/export original mientras Application lo consuma, actualizar primero el consumidor y retirar después sólo lo que quede sin uso. El diff actual de Foundation elimina exports S3 aún consumidos y modifica la política del bucket. Con Drive configurado y validado, desplegar primero Application con `cdk deploy PukStagingApplication --exclusively` desde `infra/cdk`, sin actualizar Foundation como dependencia; no empezar con `deploy --all`. Después de liberar imports y validar la aplicación, actualizar Foundation. No borrar datos como atajo; volver a comprobar versiones y marcadores de borrado antes de considerar el bucket vacío como candidato a limpieza.

No se deben renombrar construct IDs ni mover recursos persistentes sin revisar primero `pnpm infra:diff`: un cambio de logical ID puede provocar reemplazo. Tampoco se debe eliminar un export del stack foundation mientras Application lo consuma.

Identidades y configuración de ejecución

- ECS inyecta credenciales de RDS desde su secreto generado al iniciar `migration` y `api`.
- El inventario del 2026-10-05 observó el secreto runtime `puk/staging/runtime` en Secrets Manager, `us-east-2`; staging está actualmente eliminado/no desplegado.
- La configuración desplegada histórica contenía referencias de Sheets. La configuración CDK local de Drive agrega `GOOGLE_DRIVE_CLIENT_EMAIL`, `GOOGLE_DRIVE_PRIVATE_KEY` y `GOOGLE_DRIVE_SHARED_DRIVE_ID`; requiere configurar y verificar una identidad y una unidad compartida de pruebas.
- AWS staging está eliminado/no desplegado. Gmail funciona localmente sólo con las variables `GOOGLE_GMAIL_CLIENT_EMAIL`, `GOOGLE_GMAIL_PRIVATE_KEY` y `GOOGLE_GMAIL_SENDER_EMAIL`, más delegación Workspace para `https://www.googleapis.com/auth/gmail.send`; el remitente es tanto el usuario suplantado como el encabezado `From`, y cambiarlo requiere reiniciar la API.
- El CDK de Application no mapea `GOOGLE_GMAIL_*` al entorno de la tarea. Gmail no puede habilitarse en AWS agregando únicamente los valores al secreto: primero se debe actualizar la IaC/task secret mapping y desplegar. El envío de reportes es manual, no automático.
- Las identidades de Sheets y Drive pueden ser distintas; no inferir ni reutilizar sus claves automáticamente. Permisos IAM de Google Cloud no sustituyen compartir los recursos de Workspace.
- La IaC define el rol OIDC `puk-staging-github-deploy`, limitado a `main` del repositorio configurado.
- Las variables no sensibles se definen en el task definition; nombres de pestañas y `STORAGE_PROVIDER` no cambian editando sólo los IDs en Secrets Manager.

El procedimiento completo de cuentas de servicio, IDs, permisos y actualización parcial está en Configurar Sheets y Drive. No copiar claves o archivos JSON a chat, logs, tickets ni Git.

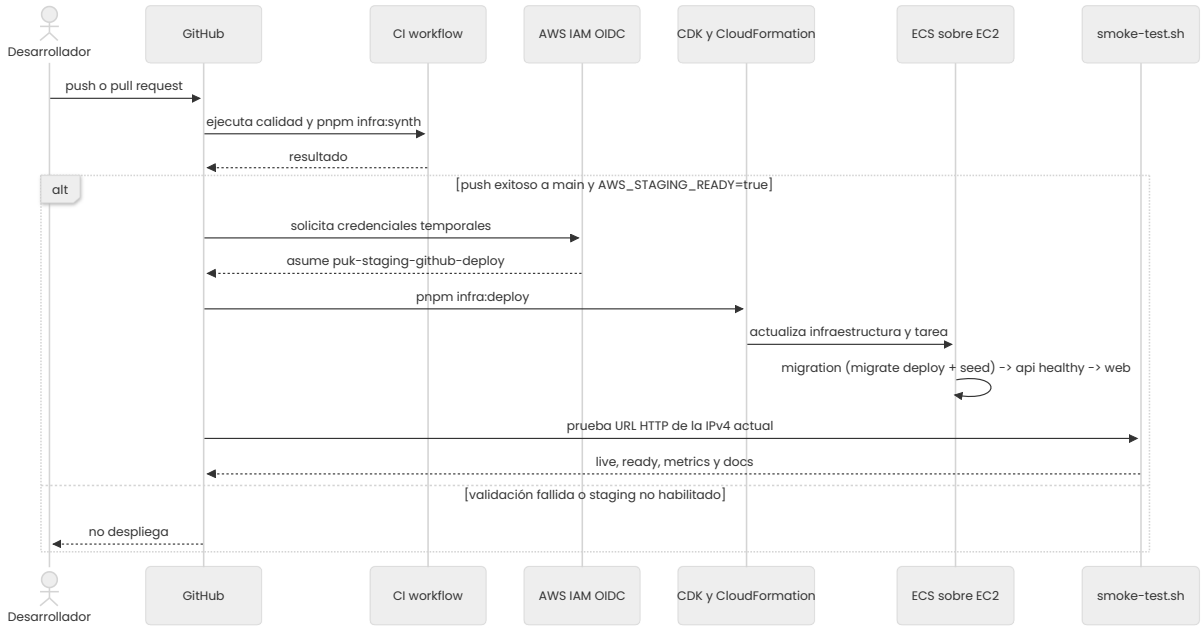
Observabilidad

Esta sección describe los checks y logs definidos por la IaC y la aplicación; no valida por sí sola el acceso a Google.

- ECS Container Insights está deshabilitado en el cluster `puk-staging` para evitar métricas adicionales; permanecen disponibles los health checks y logs de CloudWatch.
- El host usa ECS-optimized Amazon Linux 2023 x86_64, no expone SSH y acepta HTTP público sólo por TCP/80. No tiene Elastic IP; si la instancia se reemplaza, cambia su IPv4 pública de acceso y egreso.
- `GET /health/live` comprueba que el proceso de API responda.
- `GET /health/ready` comprueba sólo conectividad con PostgreSQL y se usa en el smoke test. No prueba Sheets, autenticación de Google ni permisos de Drive; validarlos por separado.
- `GET /metrics` expone métricas Prometheus, pero el CDK no crea un scraper, alarmas ni dashboards.
- `GET /health/metrics` ofrece un snapshot JSON desde la aplicación.
- Los logs de migración, API y web se envían a grupos separados de CloudWatch Logs con retención de un mes.
- Cada petición recibe un `requestId`; puede propagarse uno válido desde el cliente.

Integración y despliegue continuo

> **Workflow declarado:** `.github/workflows/deploy-staging.yml` corre después de un CI exitoso sobre `main` sólo si `AWS_STAGING_READY == 'true'`. El valor actual no se verificó ni modificó. Resolver configuración de Google y exports entre stacks antes de habilitar despliegues automáticos.



El workflow `.github/workflows/ci.yml` instala dependencias, valida Prisma, ejecuta lint, typecheck, pruebas, validación OpenAPI, build, `infra:synth`, auditoría y E2E. Después de un CI exitoso sobre `main`, `.github/workflows/deploy-staging.yml` hace checkout de `workflow_run.head_sha`, ejecuta `pnpm infra:deploy` y smoke-tests sólo si la variable de repositorio `AWS_STAGING_READY` vale exactamente `true`. La concurrencia está serializada y no cancela un despliegue en curso. Aunque éste sea el flujo declarado, staging actualmente no está desplegado.

Los assets Docker de CDK se construyen y publican mediante el bootstrap de CDK. El repositorio no define ECR repositories ni tags de aplicación explícitos.

El workflow de despliegue hace checkout de `github.event.workflow_run.head_sha`, por lo que CI despliega la revisión que pasó las validaciones. Un despliegue manual de assets o de CDK debe registrar explícitamente el SHA de Git, la revisión de ECS y el operador para conservar la trazabilidad de release.

Actualización de secretos y operación

Para cambiar IDs o una cuenta de servicio, usar la consola de Secrets Manager y editar únicamente los campos de Google de `puk/staging/runtime`, preservando todos los campos `JWT_*`, `SEED_*` y cualquier otro valor existente. La guía de staging explica el mapeo del JSON y los permisos de los documentos y la unidad compartida.

No usar `configure-staging-secrets.mjs --overwrite` para cambios parciales: reemplaza el contenido completo y regenera claves JWT y contraseñas del seed. El comando sin `--overwrite` rechaza el secreto existente y sólo es apropiado para creación inicial. Nunca mostrar claves en terminal, chat, tickets o logs. Una nueva versión del secreto no actualiza tareas ya iniciadas: requiere despliegue nuevo. La primera activación de Drive también necesita el task definition actualizado desde IaC, no sólo reiniciar la revisión 14.

Con una sesión y destino autorizados, estos diagnósticos son de sólo lectura; no copiar información sensible que pudiera aparecer en logs:

```
aws cloudformation describe-stacks --region us-east-2 --stack-name PukStagingApplication
aws ecs describe-services --region us-east-2 --cluster puk-staging --services <service-name>
aws ecs list-tasks --region us-east-2 --cluster puk-staging --service-name <service-name>
aws logs tail /puk/staging/migration --region us-east-2 --since 30m
aws logs tail /puk/staging/api --region us-east-2 --since 30m
TARGET_URL="http://<ipv4-publica-actual>" ./infra/scripts/smoke-test.sh
```

Consultar siempre la IPv4 actual con el procedimiento de la guía de staging. Antes de desplegar Drive, verificar lectura de las pestañas de pruebas y metadatos de la unidad compartida con las cuentas de servicio mediante un mecanismo seguro. Después, ejecutar smoke y pruebas funcionales con archivos y filas sintéticos. El smoke sólo comprueba `/health/live`, `/health/ready`, `/metrics` y `/docs/`; no certifica la integración con Google.

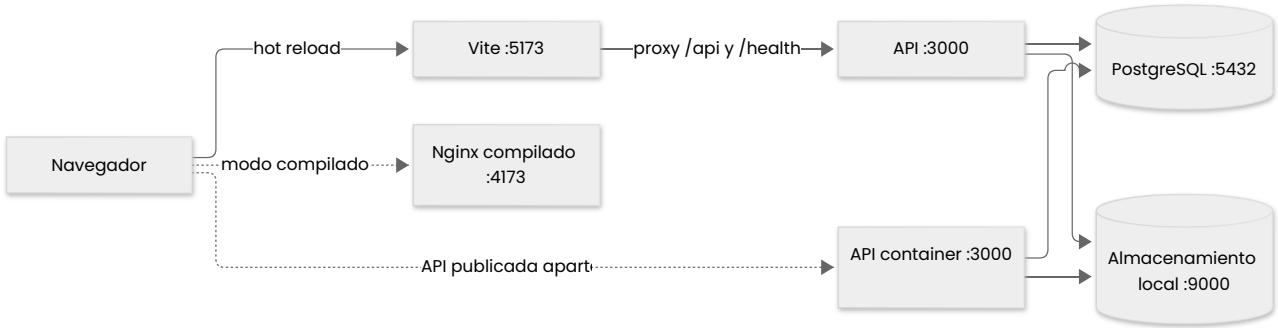
Si falla la migración, la API no inicia; conservar logs y evaluar la migración antes de reintentar. El circuit breaker de ECS puede revertir la aplicación, pero no restaura automáticamente RDS. `RETAIN` protege algunos recursos al eliminar un stack; no es un backup. Usar sólo respaldos verificados y autorización operativa para restaurar o decidir el destino de datos.

RDS, EC2, EBS, la IPv4 pública y los servicios asociados pueden generar costos incluso sin tráfico. La limpieza debe limitarse a recursos sin consumidores, después de validar Drive y revisar los exports entre stacks; no eliminar datos o el bucket para resolver una actualización.

Desarrollo local

Servicios Docker Compose

Servicio	Imagen o proceso	Puerto predeterminado	Persistencia
`web`	Nginx + Vue compilado	4173 externo / 80 interno	No
`api`	Node.js 24 + Express	3000	No
`postgres`	PostgreSQL 18	5432	Volumen `postgres-data`
`minio`	MinIO	9000 API, 9001 consola	Volumen `minio-data`
`minio-init`	Cliente `mc`	No aplica	Crea el bucket privado



En hot reload, el proxy de Vite envía `/api` y `/health` a `VITE_API_PROXY_TARGET`. En ECS, Nginx y API comparten tarea y `127.0.0.1:3000` es correcto. En Docker Compose están en contenedores distintos, pero la configuración Nginx también apunta a `127.0.0.1:3000`; por ello el proxy `/api` del contenedor web compilado no alcanza actualmente a `api`. Hasta corregir esa divergencia, use hot reload o el puerto publicado de la API para validar el modo local.

Inicio rápido

```
pnpm install
cp .env.example .env
docker compose up -d postgres minio minio-init
pnpm prisma:migrate
pnpm prisma:seed
pnpm dev
```

Para construir todos los contenedores:

```
docker compose up --build -d
```

Los Dockerfiles son multi-stage. La imagen de API se ejecuta como usuario `node`; el frontend se construye con Vite y se sirve desde Nginx. Las versiones admitidas son Node.js 24 a 26 y pnpm 11.

Variables de entorno

Grupo	Variables principales
HTTP	`PORT`, `APP_ORIGIN`, `API_ORIGIN`, `TRUST_PROXY`
Base de datos	`DATABASE_URL`, `DIRECT_URL` o variables `PUK_DATABASE_*` en ECS
JWT	Claves pública y privada, `JWT_ISSUER`, `JWT_AUDIENCE`, TTLs
Cookies	`COOKIE_DOMAIN`, `COOKIE_SECURE`, `COOKIE_SAME_SITE`
S3 (legado/local)	Endpoint, región, bucket, credenciales locales y path style; revisión histórica 14 usó S3, Drive está en IaC
Drive	Cuenta de servicio, clave privada e ID de unidad compartida (`GOOGLE_DRIVE_*`)
Gmail	Cuenta de servicio, clave privada y buzón suplantado (`GOOGLE_GMAIL_*`); no mapeado por CDK
Sheets	Spreadsheet, cuenta de servicio, clave privada y pestañas
Límites	Tamaños de evidencia y firma, TTL de URLs, caché
Resiliencia	Timeouts, reintentos, circuit breaker y polling
Seed	Correos y contraseña inicial

La API valida su configuración al arrancar. Los valores `development-only` activan adaptadores simulados fuera de producción y nunca deben usarse en producción.

Procesos periódicos

- `SyncService` consulta el outbox cada `OUTBOX_POLL_INTERVAL_MS`, procesa hasta 20 operaciones y aplica backoff exponencial con jitter.
- `ReportsService` procesa hasta dos PDFs por ciclo y recupera trabajos interrumpidos después de diez minutos.
- Ambos intervalos usan `unref()` y se cancelan durante el apagado ordenado.
- Con `desiredCount: 1` hay una sola instancia de cada worker. El modelo debe revisarse antes de escalar la cantidad de tareas para evitar trabajo duplicado.

Operación y validación

```
pnpm typecheck
pnpm lint
pnpm test
pnpm build
pnpm prisma:validate
pnpm docs:validate
pnpm infra:synth
pnpm infra:diff
```

Antes de desplegar cambios CDK se debe ejecutar `synth` y revisar `diff`, especialmente para RDS, S3, VPC y referencias entre stacks. Hotswap y express mode no forman parte del flujo y no deben usarse en producción.

La guía operativa y el primer despliegue están en staging-and-deployment.md. Los scripts `infra/scripts/backup.sh` y `restore.sh` realizan respaldo lógico y restauración de PostgreSQL, pero no están conectados al workflow de AWS; su programación, almacenamiento externo y prueba periódica son responsabilidad operativa.

Evaluación para producción

La arquitectura declarada de staging prioriza costo y simplicidad. AWS staging no está desplegado; Drive sigue pendiente de validación antes de un redeployment. La tabla compara la configuración objetivo con el trabajo previo a producción:

Pilar	Configuración objetivo de IaC	Trabajo pendiente
Seguridad	RDS cifrado; Drive privado pendiente; HTTP público; OIDC en CI; el inventario histórico observó S3	Configurar dominio y TLS; limitar ingreso; definir controles de Workspace; agregar WAF según riesgo; auditar permisos con cdk-nag

Pilar	Configuración objetivo de IaC	Trabajo pendiente
Confiabilidad	Dos AZ para subredes; rollback de ECS; política de retención prevista para RDS	RDS Multi-AZ; alarmas; estrategia de migración única; restauraciones probadas
Rendimiento	Sin CDN ni caché de borde	Autoscaling ECS, pruebas de carga y dimensionamiento de RDS
Costos	Sin NAT; host EC2 único con IPv4 pública y RDS micro	Presupuestos y alarmas de costo; revisar transferencia e IPv4 públicas
Operación	Logs por contenedor y smoke test	Dashboards, alarmas, trazas, runbooks y objetivos SLO
Sostenibilidad	Contenedores dimensionados explícitamente	Medir utilización y ajustar CPU, memoria y retenciones

Referencias oficiales AWS

- Conectar aplicaciones ECS a Internet desde subredes privadas
- Periodo de retención de backups automatizados de RDS
- Buenas prácticas de desarrollo y despliegue con AWS CDK